SOP-UQ 快速上手

服务协议 3.0 · 主数据格式 2.1 · 考生版

本文说明如何连接考官服务，并用测试 notebook `scripts/test_example.ipynb` 走完整个流程。题面（完整规则）见 `docs/problem_statement.pdf`。

1. 安装

在 `sop_uq_candidate/` 目录的 bash 或 zsh 中（需要 conda；Windows 请用 WSL）：

```
source conda_env.sh      # 首次创建 conda 环境 sii_uq（几分钟），以后只激活
export SOP_EXAMINER_URL=http://<考官地址>:8801 SOP_EXAMINER_TOKEN=<本队试用 token>
export SOP_MODEL_URL=http://<模型地址>:18080 SOP_MODEL_TOKEN=<模型 token>
jupyter notebook scripts/test_example.ipynb
```

`conda_env.sh` 按 `environment.yml` 安装 Python 3.12、本包与 Jupyter，环境损坏时自动修复；每个新终端先运行它。不用 conda 的安装方法见 `README.md`。

每队有两个 token：**试用 token** 用于跑 notebook 示例、熟悉流程，在测试题上只能做少量工具调用和 oracle 读取，提交只检查格式、不评分、不占次数，也不影响正式台账；**正式 token** 用于正式运行和提交。

2. 数据

下表列出 `data/` 下的全部文件，路径均相对于考生包目录。

文件	内容
`data/train/train.jsonl`	训练题输入与标签，用于学习风险估计方法
`data/train/cal_fit.jsonl`	校准题输入与标签，用于校准预测概率
`data/train/cal_select.jsonl`	输入与标签，用于选择查询策略和转人工阈值
`data/train/dev.jsonl`	验证题输入与标签，用于调参和误差分析
`data/test/cases.jsonl`	测试题输入，不含标签
`data/test/initial_M9.jsonl`	Qwen 的固定初始答案、工具调用记录和共享采样结果
`data/manifest.json`	数据版本、各 JSONL 文件的行数和 SHA-256 校验值

`.jsonl` 文件采用 JSON Lines 格式，每行一个 JSON 对象。测试输入与初始轨迹按 `case_id` 对应，不按行号配对。交互题名单由 `examiner.spec()["interactive_ids"]` 给出，题目文件不单独标注。业务规则和工具说明见 `src/baselines/resources/`，字段的完整说明见 `README.md`。

3. 工具与 oracle

`examiner.episode(case_id)` 打开一题，`examiner.tool(episode_id, name, arguments, mode)` 调用业务工具，`examiner.oracle(episode_id, field, phase)` 读取真值。每题可用的工具及参数见 `episode["view"]["tools"]`：各域 `toolspecs.json` 中的工具，外加 SOP 要求的业务动作（见各域 `amendment.txt`）。

业务	工具（有噪声的读数）	oracle 字段（phase）
危险品	四个评分工具 `calculate_sds_label_score`、`calculate_handling_score`、`calculate_transportation_score`、`calculate_disposal_score`，四项评分都有噪声	`sds`、`handling`、`transportation`、`disposal`（`assessment`）
飞机部件检查	`VerifyAircraftClearance`、`VerifyMechanicalComponents`、`VerifyElectricalSystems`、`CrossCheckSpecifications` 的结果有噪声；三个 `Report...` 工具返回已有的报告结论（无噪声）；`updateMaintenanceRecord` 记录维修记录	前四个工具的结果字段（`assessment`）
客服排障	按 SOP 顺序：验证账户、认证、建会话与工单、查账户状态（停用时先走付款分支 `transferAccountsPayable`、`checkPaymentStatus`、`checkAccountSuspensionStatus`）、查区域故障、`performTechnicalDiagnostics` 诊断、`executeTroubleshooting` 排障，最后 `updateTicket` 或 `createEscalation`；诊断和排障的 `latency`、`jitter`、`bandwidth` 有噪声	这三项：`pre_repair`（诊断后）或 `post_repair`（排障后）

`mode`：`default` 读主数据源，可能是缓存的旧记录；`refresh`、`remeasure` 重新读主数据源，同一来源的重复读数可能相关；`independent` 读第二个数据源，它与主数据源不一定相互独立。普通调用 0.01，有噪声工具的 `independent` 读数 0.03，oracle 每个字段 0.10，同一字段再读免费；顺序不对时返回 `phase_denied`。有标签题可以不限次数地重新打开，每次都是新的练习。测试题中只有 72 道交互题可以继续调用。读取测试题的 oracle 之前须先提交一次结果；本队第一次读取测试题 oracle 后，问题 1（查询前风险 `p_initial_error`）的成绩不再更新，此后的提交只计入问题 2（交互题）。

4. 测试 notebook 的八节

用试用 token 从上到下运行各单元格。前四节各需几秒；第 5 节在有标签题上运行 Qwen，平均每题约 10 秒（客服题约 15 秒；多队共用模型服务时更久）；第 6 节不调用模型；第 7 节只在读取了 oracle 的交互题上调用模型（正式 token 约 1 分钟）。

	内容	你会看到
1	连接	费用、单题限制、360 道测试题中的 72 道交互题、剩余提交次数
2	数据	各划分题数、一道题的输入与标签、冻结初始答案
3	手动调用工具与 oracle	回执、噪声读数（两个数据源）、oracle 真值、累计费用
4	一次 baseline 作答	agent 的答案、自报风险、四次采样的分歧率
5	采集轨迹并拟合	一次采集 `LIMIT` 题/划分：初始答案、四次采样、一次 oracle 与继续作答；拟合初始与查询后风险
6	问题 1：测试题风险并提交	不读 oracle 的 360 题 `results.csv`；提交后的三项风险与决策代价、剩余次数（试用 token 只检查格式）
7	问题 2：oracle 与决策并提交	交互题读取 oracle、继续作答、`AUTO`/`DEFER`；正式 token 须先设 `DONE_WITH_PROBLEM_1 = True`
8	你自己的方法	改写问题 1 的风险后检查文件，准备下一次提交

第 5 节的 `LIMIT = 4` 只是快速试跑；正式方法应设 `LIMIT = None` 采集全部 660 道有标签题（独占一张 GPU 约 2 小时，多队共用模型服务时更久，请尽早开始）。

另一个 notebook `scripts/baseline_example.ipynb` 在同样的轨迹上离线比较六种风险方法和转人工阈值；它的 `RUN` 默认指向第 5 节的 `.runs/notebook/collect`，用命令行采集时改为 `.runs/collect`。

5. 命令行：同样的流程

```
# 一次采集有标签题：初始轨迹 + 每题一次 oracle 后的继续作答
# （--limit 为每个划分的题数；不写则全部）
sop-candidate collect --limit 4 --policy fixed --output .runs/collect
# 拟合初始风险与查询后的两项风险
sop-candidate fit --trajectories .runs/collect --output .runs/cal.json
# 问题 1：不读 oracle 的测试运行，检查并提交
sop-candidate run --calibrator .runs/cal.json --uq calibrated_disagreement \
    --policy threshold --output .runs/problem1
python scripts/run_test.py .runs/problem1/results.csv
# 问题 2：读取测试题 oracle（问题 1 随即结束），继续作答并提交
sop-candidate run --calibrator .runs/cal.json --uq calibrated_disagreement \
    --policy fixed --output .runs/problem2
python scripts/run_test.py .runs/problem2/results.csv
python scripts/run_test.py --submissions
# 有标签题的结果可在本地评分，不占次数
python scripts/run_test.py .runs/collect/results.csv --local
```

每个输出目录可中断续跑；改了方法设置时请换一个新的 `--output`。

6. 结果文件 results.csv

每道测试题一行，列为

```
case_id, initial_answer_json, p_initial_error, mode,
final_answer_json, p_initial_error_after_queries, p_final_error
```

- 答案为 JSON 对象，无答案留空；三项概率为 [0,1] 内的数值，无法产生时留空。
- 非交互题只填前三列；交互题的 `mode` 为 `AUTO` 或 `DEFER`。
- `initial_answer_json` 必须等于冻结的初始答案。
- 用 `test_client.write_results` 写文件，用 `test_client.results.check_results` 在提交前检查。

7. 提交与成绩

每次有效提交立即返回完整评分：三项风险（Brier、NLL、ECE、AP、AURC，按业务域和三域平均）与交互题的决策代价。格式错误会被拒绝；与已评分文件完全相同的文件直接返回原报告；两者都不占次数。每队最多 5 次：问题 1 取本队第一次读取测试题 oracle 之前 p_i 的三域平均 Brier 最好的一次，问题 2 取所有提交中决策代价最低的一次（都是越低越好）。测试题的工具和 oracle 费用记在本队台账中，重跑不会清零。试用 token 的提交只检查格式，不评分、不占次数。

评分反馈用于评估自己的方法；考官信任各队，不得用它推断测试标签（例如逐题改变风险值比较分数）。试用 token 读到的测试题真值只用于熟悉流程，不得用于正式提交。比较不同方法请在有标签题上进行：测试题台账由各方法共用。

8. 常见错误

错误码	含义与处理
`unauthorized`	考官或模型的 token 不对或没设置（`SOP_EXAMINER_TOKEN`、`SOP_MODEL_TOKEN`）
`noninteractive_case`	这道测试题不允许再调用工具或 oracle
`submit_first`	读取测试题的 oracle 之前须先提交一次结果
`trial_quota`	试用 token 在测试题上的工具调用或 oracle 读取已用完；正式运行请换正式 token
`phase_denied`	业务步骤顺序不对，例如客服未诊断就查指标
`budget_exhausted`, `tool_limit`, `oracle_limit`	本题已用完预算 2.00、24 次工具调用或 16 次 oracle
`invalid_results`	文件未通过检查（格式、取值，或初始答案不等于冻结答案），消息写明行或 `case_id`；不占次数
`no_tries_left`	本队的提交次数已用完
`model_busy`	共享模型繁忙；客户端会自动等待重试